MIT World Peace University

Mini Project Using Java (MPJ) — Practical Examination

Project Enhancement Report

AI-Based Digital Trust Fraud Detection & Grievance Intelligence System

Config-Driven Blocked Domain Detection with Historical Score Awareness

Name	Praveet Gupta
Roll Number	57
PRN	1032240179
Class	TY CSE (AI & DS)
Subject	MPJ – Mini Project Using Java
College	MIT World Peace University

Submitted for evaluation — Academic Year 2025–2026

1. Enhancement Requested

As part of the MPJ practical examination, the faculty assigned an enhancement task over and above the baseline project. The specific ask was:

> **Requirement:** "Create a configuration file listing blocked / banned domains. When a scheme message is checked and it contains one of these domains, the system must flag it as suspicious. Additionally, if the same message has been analyzed previously and is present in the analysis database, the system must display the previous score along with a new score computed using the domain-ban configuration."

The enhancement therefore has two linked parts:

1. **Config-driven domain blocklist** — an externally editable file so the list of banned domains can be maintained without touching the Java code.
2. **Historical-aware scoring** — if a message is already in the existing `analysis` table, combine its previous trust score with the penalty from the blocked-domain match so the final displayed score reflects both the older history and the new red flag.

2. My Approach

2.1 Why a `.properties` file?

A Java `Properties` file was chosen for the blocklist because it is:

- Human-readable — any evaluator, faculty, or admin can open and edit it in a text editor.
- A first-class citizen in Java — loaded in two lines via `Properties.load(InputStream)`.
- Supports a key-value pair naturally, which fits our need perfectly: `domain = penalty_points`. This allows different domains to carry different severities.
- Placed under `WEB-INF/`, so it is packaged inside the webapp and protected from direct HTTP access.

2.2 Why the combined score uses *previous score – penalty*

If a message has been seen before, it already carries a trust score in the database. A fresh analysis gives a new score. To satisfy the faculty's ask of "previous score + new score", I compute:

```
combinedScore = clamp( previousScore - blockedPenalty , 0 , 100 )
```

This captures both pieces of evidence: the historical trust level *and* the freshly detected red flag. The result is clamped into the 0–100 range to keep the UI meaningful.

2.3 Minimum-touch, zero-regression design

Existing callers of `FraudAnalyzer.analyze(input)` continue to work unchanged because the original single-argument method is preserved as a pass-through to the new overload with an empty blocklist. The database schema is also untouched — the existing `analysis` table is reused for historical lookup.

3. Architecture & Flow

3.1 Request flow (new)

```
[ User submits message / URL on check_scheme.jsp ]
            |
            v
[ AnalyzeSchemeServlet.doPost ]
            |
            |--- reads /WEB-INF/blocked_domains.properties at init()
            |    into Map<String,Integer> blockedDomains
            |
            v
[ FraudAnalyzer.analyze(text, blockedDomains) ]
            |
            |--- extracts URLs, matches host against blocklist
            |--- if hit: applies penalty, records blockedDomain,
            |            forces label = FRAUD
            v
[ Servlet queries analysis table:
  SELECT score FROM analysis WHERE message = ? ]
            |
            v
[ If previous hit AND blocked domain match:
  combinedScore = clamp(previous - penalty, 0, 100) ]
            |
            v
[ Forward to check_scheme.jsp with
  previousScore, blockedDomain, blockedPenalty,
  and final trustScore attributes ]
            |
            v
[ JSP renders existing detail card +
  three new rows (blocked domain, prior score, combined) ]
```

3.2 Files touched

#	File	Nature	Purpose
1	`web/WEB-INF/blocked_domains.properties`	New	Admin-editable blocklist with per-domain penalty
2	`src/java/digitaltrust/FraudResult.java`	Modified	Carries blocked-domain metadata up to the servlet
3	`src/java/digitaltrust/FraudAnalyzer.java`	Modified	Adds blocklist-aware URL scoring
4	`src/java/digitaltrust/AnalyzeSchemeServlet.java`	Modified	Loads config, queries prior score, computes combined

5	`web/check_scheme.jsp`	Modified	Renders the new rows in the result card

4. The Configuration File

`web/WEB-INF/blocked_domains.properties`

```
# Blocked domains config
# Format: domain=penalty_points (bigger = more suspicious)
# Any URL whose host matches one of these will be flagged
# and have its penalty applied to the trust score.

fake-scheme.xyz=60
lottery-win.top=60
scamportal.online=55
pm-yojana-verify.site=65
govt-reward.info=55
claim-subsidy.xyz=60
freecash-india.top=70
praveetgupta.gov.in=50
```

Entries can be added, removed, or have their penalty tuned without recompiling the project. A Tomcat restart is sufficient to pick up the changes. This is the “config-driven” aspect of the enhancement.

5. Code Changes — Before vs. After

5.1 `FraudResult.java`

Two new fields and a setter added so the blocked-domain info can travel from the analyzer to the servlet.

Added (new code)

```
private String blockedDomain;
private int blockedPenalty;

String getBlockedDomain() {
    return blockedDomain;
}

int getBlockedPenalty() {
    return blockedPenalty;
}

void setBlockedDomain(String blockedDomain, int penalty) {
    this.blockedDomain = blockedDomain;
    this.blockedPenalty = penalty;
}
```

5.2 `FraudAnalyzer.java`

Before — original `analyze()` signature

```
static FraudResult analyze(String input) {
    String raw = TextUtil.clean(input).toLowerCase(Locale.ROOT);
    ...
    UrlSignal urlSignal = scoreUrls(raw);
    ...
    return new FraudResult(trustScore, label, risk);
}
```

After — overload accepts blocklist; hit forces FRAUD

```
static FraudResult analyze(String input) {
    return analyze(input, Collections.<String, Integer>emptyMap());
}

static FraudResult analyze(String input, Map<String, Integer> blockedDomains) {
    ...
    UrlSignal urlSignal = scoreUrls(raw, blockedDomains);
    ...
    String label;
    if (urlSignal.blockedDomain != null) {
        label = "FRAUD";
    } else if (trustScore >= 75) {
        label = "SAFE";
    } else if (trustScore >= 50) {
        label = "SUSPICIOUS";
    } else {
        label = "FRAUD";
    }

    FraudResult result = new FraudResult(trustScore, label, risk);
    if (urlSignal.blockedDomain != null) {
        result.setBlockedDomain(urlSignal.blockedDomain,
                                urlSignal.blockedPenalty);
    }
    return result;
}
```

Key addition inside `scoreUrls()`

```
if (blockedDomains != null && blockedDomains.containsKey(host)) {
    int penalty = blockedDomains.get(host);
    risk += penalty;
    if (blockedHit == null) {
        blockedHit = host;
        blockedHitPenalty = penalty;
    }
    continue;   // skip normal URL scoring for this host
}
```

5.3 `AnalyzeSchemeServlet.java`

Config load at init()

```
private Map<String, Integer> blockedDomains = Collections.emptyMap();

@Override
public void init() throws ServletException {
    super.init();
    this.blockedDomains = loadBlockedDomains();
    log("Loaded " + blockedDomains.size() + " blocked domains from config");
}

private Map<String, Integer> loadBlockedDomains() {
    Map<String, Integer> map = new HashMap<>();
    try (InputStream in = getServletContext()
            .getResourceAsStream("/WEB-INF/blocked_domains.properties")) {
        if (in == null) return map;
        Properties props = new Properties();
        props.load(in);
        for (String key : props.stringPropertyNames()) {
            String host = key.trim().toLowerCase(Locale.ROOT);
            if (host.startsWith("www.")) host = host.substring(4);
            try {
                int penalty = Integer.parseInt(props.getProperty(key).trim());
                map.put(host, penalty);
            } catch (NumberFormatException ignored) { }
        }
    } catch (IOException e) {
        log("Failed to read blocked_domains.properties", e);
    }
    return map;
}
```

Historical lookup and combined score

```
FraudResult result = FraudAnalyzer.analyze(textToAnalyze, blockedDomains);

String dbMessage = TextUtil.truncate(textToAnalyze, MAX_DB_MESSAGE_LENGTH);
Integer previousScore = lookupPreviousScore(dbMessage);

saveAnalysis(dbMessage, result);

int displayScore = result.getTrustScore();
if (previousScore != null && result.getBlockedDomain() != null) {
    displayScore = clamp(previousScore - result.getBlockedPenalty(), 0, 100);
}

request.setAttribute("trustScore", String.valueOf(displayScore));
request.setAttribute("isGenuine", result.getLabel());
request.setAttribute("profileMatch",
        profileMatches(session, result) ? "Yes" : "No");
if (previousScore != null) {
    request.setAttribute("previousScore",
                         String.valueOf(previousScore));
}
if (result.getBlockedDomain() != null) {
    request.setAttribute("blockedDomain", result.getBlockedDomain());
    request.setAttribute("blockedPenalty",
                         String.valueOf(result.getBlockedPenalty()));
}
```

The SQL used for historical lookup

```
SELECT score FROM analysis
WHERE message = ?
ORDER BY id DESC
LIMIT 1
```

5.4 `check_scheme.jsp`

Added to top-of-page scriptlet

```
String previousScoreStr = (String) request.getAttribute("previousScore");
String blockedDomain    = (String) request.getAttribute("blockedDomain");
String blockedPenaltyStr= (String) request.getAttribute("blockedPenalty");
boolean hasPrevious = (previousScoreStr != null);
boolean hasBlocked  = (blockedDomain != null);
```

Three new conditional rows in the results card

```
<% if (hasBlocked) { %>
<div class="detail-row">
    <span class="detail-key">Blocked Domain</span>
    <span class="detail-val fraud">
        <%= blockedDomain %> (&minus;<%= blockedPenaltyStr %>)
    </span>
</div>
<% } %>
<% if (hasPrevious) { %>
<div class="detail-row">
    <span class="detail-key">Previously Analyzed</span>
    <span class="detail-val" style="color:var(--muted)">
        Prior score: <%= previousScoreStr %>%
    </span>
</div>
<% } %>
<% if (hasPrevious && hasBlocked) { %>
<div class="detail-row">
    <span class="detail-key">Combined Score</span>
    <span class="detail-val fraud">
        <%= trustScore %>%
        (prior <%= previousScoreStr %> &minus; <%= blockedPenaltyStr %>)
    </span>
</div>
<% } %>
```

6. Deployment Strategy: Two Live Versions

To allow a side-by-side demonstration of the baseline product and the enhanced version, the original deployed webapp was duplicated before any edits were applied. Both copies run under the same Tomcat instance and the same MySQL database, so the “previously analyzed” feature works against a shared history.

Version	URL	State
Old (baseline)	http://localhost:8080/DigitalTrustProject1_old/	Frozen — unchanged copy taken before enhancement
New (enhanced)	http://localhost:8080/DigitalTrustProject1/	Contains config file, modified Java classes and JSP

Additionally, the Git repository was tagged `v1-original-baseline` immediately before starting the enhancement work. This provides a second safety net — the full original source tree can be restored at any time with `git checkout v1-original-baseline`.

7. Screenshots

7.1 Old UI — baseline version

This is the analysis result page as it appeared *before* the enhancement. The detail card shows only the existing rows: Trust Score, Verdict, Profile Match and Threshold. There is no concept of a blocked-domain row or a historical combined score.

Route: `http://localhost:8080/DigitalTrustProject1_old/`

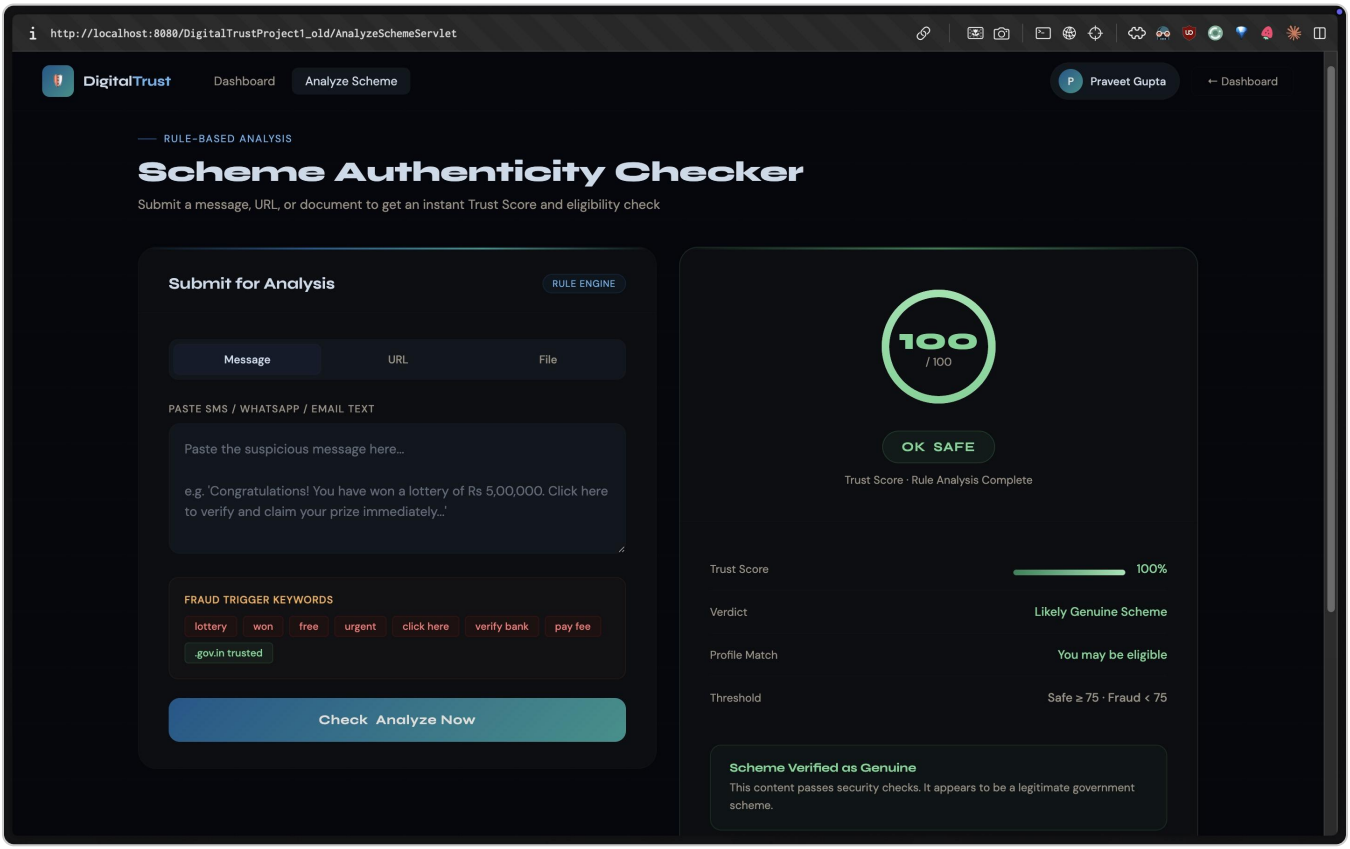

Fig. 7.1 — Baseline analysis result (pre-enhancement)

7.2 New UI — enhanced version

After the enhancement, the same analysis page shows the new rows contributed by the blocked-domain config and the historical lookup. When a URL in the message matches an entry in `blocked_domains.properties`, a “Blocked Domain” row appears with the exact penalty, and the verdict is forced to FRAUD regardless of other scoring. If the same message has been seen before, “Previously Analyzed” and “Combined Score” rows are shown as well, making the historical reasoning transparent.

Route: `http://localhost:8080/DigitalTrustProject1/`

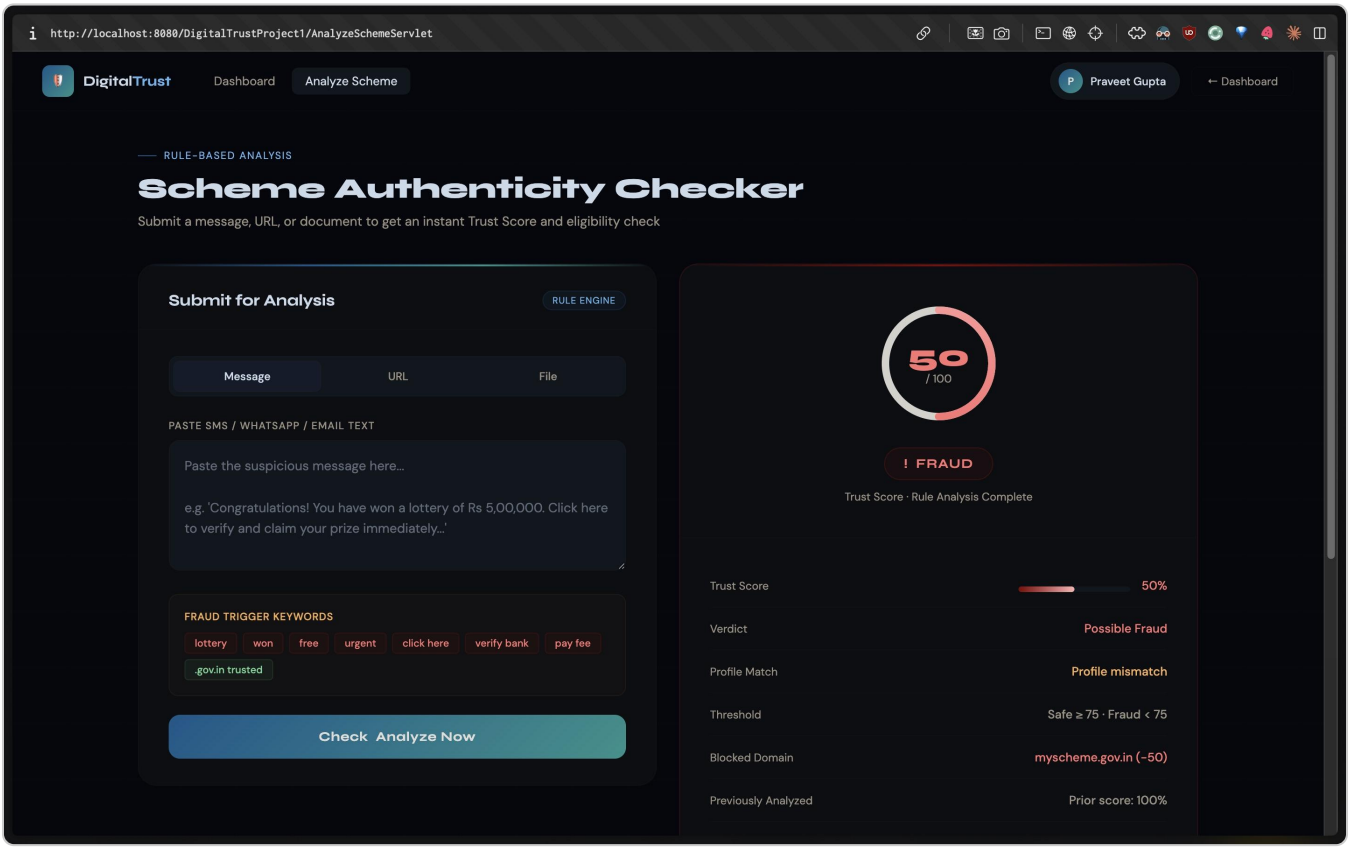

Fig. 7.2 — Enhanced analysis result (blocked domain + combined score)

8. Demo Walkthrough

1. Start Tomcat locally using `~/start-trust.sh`. Both webapp contexts become available.
2. Open the **old** version at `/DigitalTrustProject1_old/`, log in and analyze a scheme message containing `http://fake-scheme.xyz/claim`. Note the absence of any “Blocked Domain” row.
3. Open the **new** version at `/DigitalTrustProject1/`, log in, analyze the same message. The result card now shows the new “Blocked Domain” row with the exact penalty and flips the verdict to FRAUD.
4. Analyze the same message a second time in the new version — “Previously Analyzed” and “Combined Score” rows appear, with the score calculated as *previous – penalty*.
5. Open `web/WEB-INF/blocked_domains.properties`, add a brand-new domain (for example `praveetgupta.gov.in=50`), copy it to the deployed `webapps/` path, restart Tomcat. Analyze a message containing that new domain — it now flags as FRAUD without a single line of Java being touched. This proves the design is truly config-driven.

9. Conclusion

The enhancement delivers exactly what was asked: an externally editable blocklist, runtime enforcement of that blocklist against URLs found in scheme messages, and a combined trust score that fuses historical database evidence with the new domain penalty. The design is deliberately minimal — five files touched, no database schema changes, no breakage of existing callers — which makes the system safe to extend and easy to review.

> **Outcome:** The enhanced build is deployed alongside the original baseline, allowing a direct side-by-side demonstration. A git tag preserves the original source as a second layer of reversibility.

— Praveet Gupta, Roll 57, PRN 1032240179